

Building a Todo App with React Native

Teaching deck: what each file does, what happens at runtime, and why the patterns matter.

Components

Props

State

Hooks

FlatList

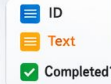
What students should understand

- How data moves from an API into the UI
- Why TodoItem and TodoScreen have different jobs
- How useState and useEffect control rendering

Building a Todo App with React Native

1. Data Type (Todo)

```
export interface Todo {  
  id: number;  
  todo_text: string;  
  is_completed: boolean;  
}
```



3. Fetching Data & State

```
const { todos, setTodos } = useState([]);  
const [loading, setLoading] = useState(true);
```

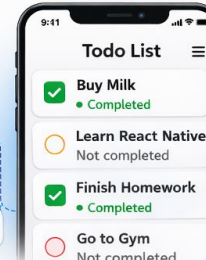
1. `getTodos()` → API Call
2. `setTodos(data)` → Save Data
3. `setLoading(false)` → Hide Loader



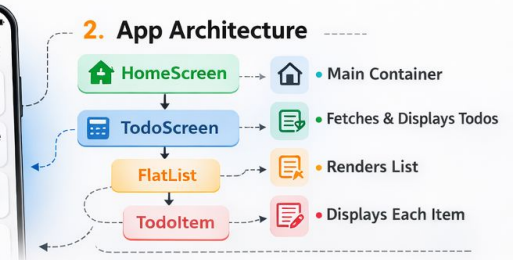
6. TodoItem Component

```
<Text style={styles.title}>{todo.todo_text}</Text>  
<Text style={styles.status}>  
  {todo.is_completed ? "Completed" : "Not completed"}  
</Text>
```

4. Loading & API Flow



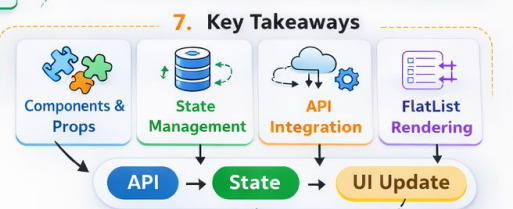
2. App Architecture



5. Rendering the List

```
<FlatList  
  data={todos}  
  keyExtractor={({item}) => item.id.toString()}  
  renderItem={({item}) => <TodoItem todo={item} />}  
>
```

7. Key Takeaways



React Native app = API + state + list rendering

1. Big picture: how the app is organized

Each file has one clear responsibility.



HomeScreen.tsx

Entry screen: places TodoScreen on the page



TodoScreen.tsx

Fetches data, stores state, renders the list



TodoItem.tsx

Displays one todo item



todoApi.ts / todo.ts

Service layer + data shape

Runtime flow

HomeScreen

TodoScreen

FlatList

TodoItem

Todo List



Buy milk

Not completed



Finish

homework

Completed



Walk the dog

Not completed

API data fills this list

Why this structure helps

- Smaller components are easier to read and test
- Screen logic stays separate from visual item layout
- The API code can change without rewriting the UI

2. Data model + API service

The app needs to know what a todo looks like and how to fetch todos.

types/todo.ts

```
1 export interface Todo {  
2   id: number;  
3   todo_text: string;  
4   is_completed: boolean;  
5 }
```

Why the type matters

- TypeScript checks that every todo has the right fields
- Autocomplete becomes better inside the editor
- It documents the data shape for the rest of the team

services/todoApi.ts

```
1 const API_URL =  
  "http://localhost:5000/todos";  
2  
3 export const getTodos = async ():  
  Promise<Todo[]> => {  
4   try {  
5     const response = await  
      fetch(API_URL);  
6     if (!response.ok) throw new  
      Error("Failed to fetch todos");  
7     const data: Todo[] = await  
      response.json();  
8     return data;  
9   } catch (error) {  
10    console.log("API Error:", error);  
11    return [];  
12  }  
13 };
```

Why the service matters

- fetch() stays outside the screen component
- Errors are handled in one place
- TodoScreen only asks for data; it does not know API details

➡ Returns Todo[]

3. TodoItem: one component for one row

TodoItem receives a single todo and turns it into visible UI.

components/TodoItem.tsx

```
1 type TodoItemProps = {
2   todo: Todo;
3 };
4
5 export default function TodoItem({ todo }:
TodoItemProps) {
6   return (
7     <View style={styles.item}>
8       <Text
style={styles.title}>{todo.todo_text}</Text>
9       <Text style={styles.status}>
10         {todo.is_completed ? "Completed" :
"Not completed"}
11       </Text>
12     </View>
13   );
14 }
```

Conditional rendering

Rendered output

☐ **Buy groceries**
Not completed

☒ **Submit assignment**
Completed

What is happening?

- The prop named todo is
- destructured from TodoItemProps
- JSX mixes layout tags with JavaScript expressions
- Styles are reused through StyleSheet.create()

Why split it out?

- FlatList can reuse the same row component for every item
- Changes to row design happen in one file. The screen stays focused on data and state, not row markup

4. TodoScreen: state + side effects

This is the smart screen that fetches data and decides what to render.

screens/TodoScreen.tsx

```
1 const [todos, setTodos] =
  useState<Todo[]>([]);
2 const [loading, setLoading] =
  useState<boolean>(true);
3
4 const fetchTodos = async () => {
5   const data = await getTodos();
6   setTodos(data);
7   setLoading(false);
8 };
9
10 useEffect(() => {
11   fetchTodos();
12 }, []);
```

What changes over time?

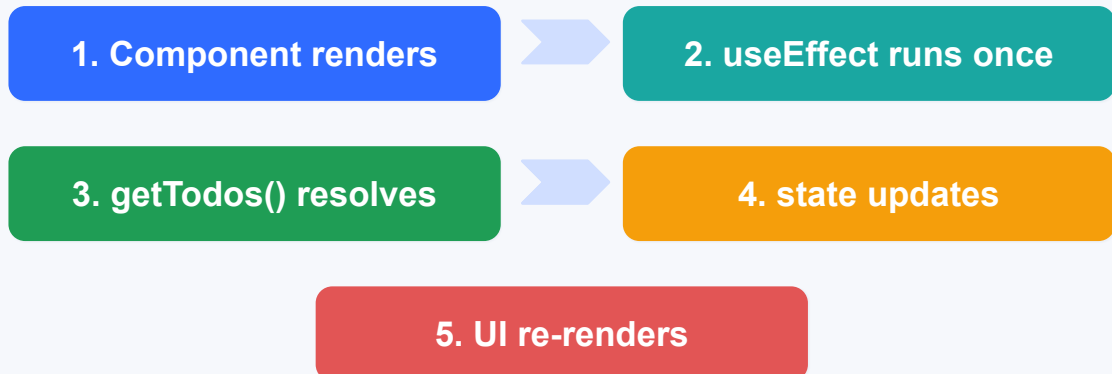
todos state

- Starts as an empty array
- Later becomes the API response
- When it changes, the list re-renders

loading state

- Starts as true
- Keeps the spinner visible
- Changes to false after data arrives

Lifecycle after the screen mounts



5. Loading state: two possible screens

Before the API responds, users see a spinner instead of an empty page.

condition
al branch

```
1 if (loading) {  
2   return (  
3     <SafeAreaView  
4       style={styles.loaderContainer}>  
5         <ActivityIndicator  
6           size="large" />  
7       </SafeAreaView>  
8   );  
9 }
```

Why this matters

- Users get immediate feedback that data is being fetched
- The app avoids showing a blank list too early
- State decides which branch of JSX is returned

Loading...



ActivityIndicator

loading =
false

Todo List

☐ Buy milk
Not completed

☒ Finish homework
Completed

☐ Walk the dog
Not completed

API data fills this list

6. FlatList turns data into many TodoItem components

React Native uses FlatList for efficient list rendering.

list
rendering

```
1 <FlatList
2   data={todos}
3   keyExtractor={({item}) =>
item.id.toString()}
4   renderItem={({ item }) => <TodoItem
todo={item} />}
5   ListEmptyComponent={<Text>No todos
found.</Text>}
6 />
```

todos array

```
[
  { id: 1, todo_text:
"Buy milk" },
  { id: 2, todo_text:
"Finish homework" },
  { id: 3, todo_text:
"Walk the dog" }
]
```

Rendered rows

☐ **Buy milk**
Not completed

☒ **Finish homework**
Completed

☐ **Walk the dog**
Not completed

Why FlatList is preferred

- It is optimized for long lists
- Each item gets a stable key from item.id
- ListEmptyComponent handles the no-data case cleanly

Teaching point

- renderItem is a function
- For each object, it returns one TodoItem component
- The UI is driven by data, not hardcoded rows

7. HomeScreen is the thin wrapper

The top-level screen simply places TodoScreen inside a full-height View.

index.tsx

```
1 import TodoScreen from
"@/screens/TodoScreen";
2 import { View } from "react-native";
3
4 export default function HomeScreen() {
5   return (
6     <View style={{ flex: 1,
backgroundColor: "#fff" }}>
7       <TodoScreen />
8     </View>
9   );
10 }
```

Composition idea

HomeScreen



TodoScreen



FlatList

TodoItem

Why keep HomeScreen simple?

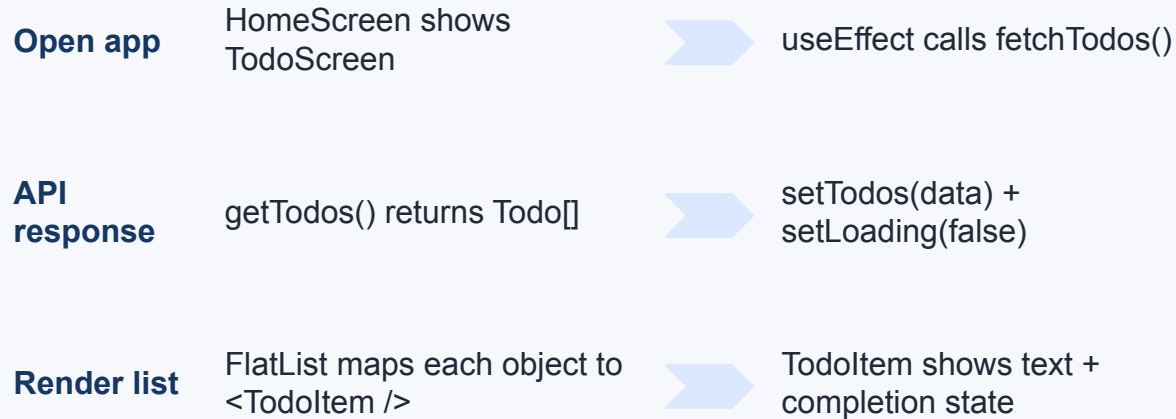
- It acts as the entry point for the route
- Layout wrappers can be changed without touching TodoScreen
- This pattern keeps responsibilities easy to explain

Key vocabulary

- Composition = building UI by nesting components
- Container = the parent that positions children
- flex: 1 = take up the available screen space

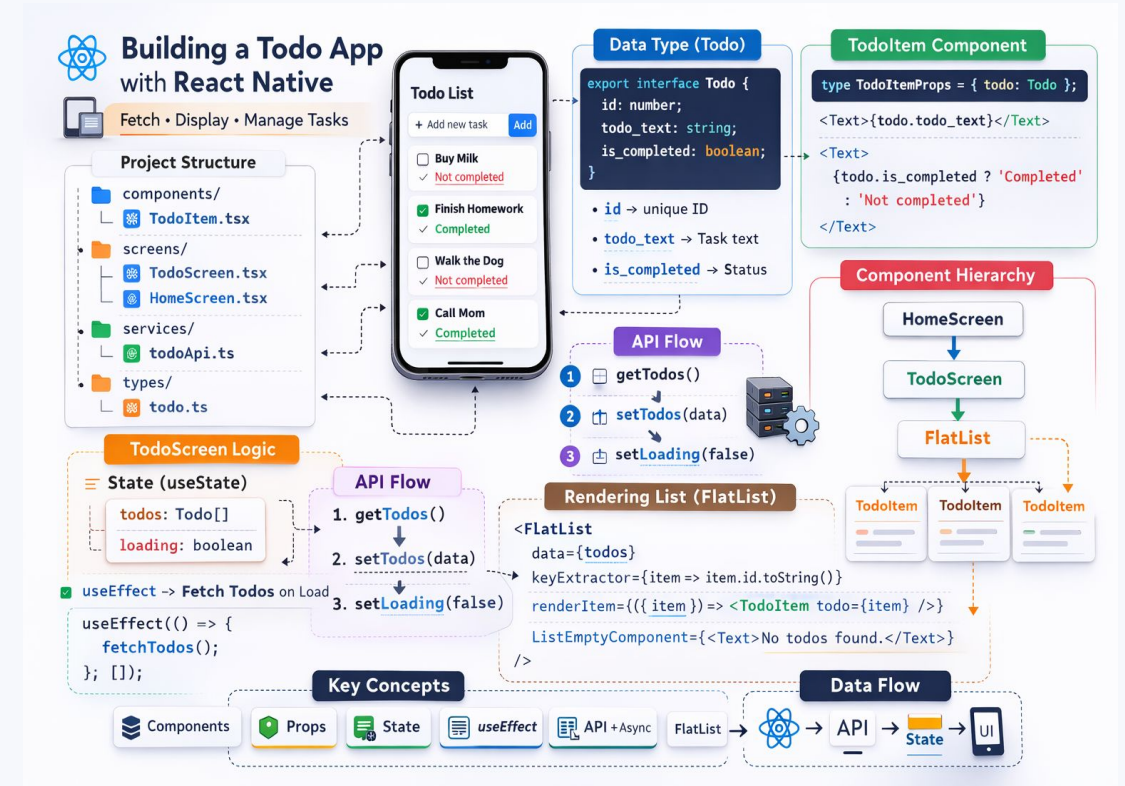
8. End-to-end lifecycle and takeaways

Students should be able to tell the story of the app from open to render.



Core lessons

- Props move data downward into child components
- State changes trigger re-renders
- useEffect is where startup fetching usually happens
- FlatList makes repeated UI data-driven



Teaching summary: API → state → list → row component
→ visible UI